Sorting Characters into Hogwarts Houses

Haltrich Thanika

Abstract

This project aims to do two things:

(1) Develop a decision tree and random forest classifier to predict which Hogwarts House each character from the downloaded dataset belongs to.
(2) Based on the ratio of non-purebloods in Slytherin from the downloaded dataset, re-cluster the characters in Hogwarts Houses in a way that reduces the ratio of non-purebloods in the Slytherin house so that it is in more accordance with the Harry Potter lore.

In the end I was able to produce a random forest classifier with a 99% accuracy score on the test dataset. Using feature-engineered columns and weighted K-Means clustering, I was able to re-cluster the characters in the dataset so that the ratio of non-purebloods in Slytherin fell from 0.67 to 0.595 and 0. Interestingly, when I was coding I made a mistake, which led me to obtain a non-pure blood ratio in Slytherin house of 0.211. This is the best result, since it is most accurate to the lore, despite the method being incorrect.

Keywords

decision tree, ensemble classifier, K-Means clustering, Weighted K-Means clustering

1 About the Dataset

The dataset I used is called 'Harry Potter Sorting Dataset' published on Kaggle by SAHITYA PALACHARLA. The link to the dataset is the following:
https://www.kaggle.com/datasets/sahityapalacharla/harry-potter-sorting-dataset/data

This is a tabular dataset which contains 1000 rows and 9 attributes. Each row represents a character, and the attributes are the following: blood status, bravery, intelligence, loyalty, ambition, dark arts knowledge, quidditch skills, dueling skills, and creativity. The target variable is 'house'. Blood status is a nominal column where a character could either be a 'pure blood', 'half blood', or a 'muggle born'. The other 8 attributes (not counting the target variable) are ordinal and on a scale of zero to ten where zero means a character does not possess that quality and ten means they possess that quality at the maximum. The target variable is nominal and contains 'Gryffindor', 'Ravenclaw', 'Hufflepuff', and 'Slytherin' (which are the four houses in Hogwarts).

With the attributes provided in this dataset, I was able to predict which house each character belongs to. Furthermore, this dataset offers room for experimentation. It is known from the Harry Potter lore that each Hogwarts house has a dominant criterion for selecting students (if a student has a high bravery score they would go to Gryffindor, if they had a high intelligence score they would go to Ravenclaw, high loyalty to Hufflepuff, and high ambition to Slytherin). However, it is also known from the lore that these attributes are not the only thing the sorting hat takes into consideration; for example, Slytherin house is prejudiced against half-bloods and muggle-borns. Therefore I was able to train the classifier algorithms using different attributes from the downloaded dataset. And lastly, after exploring the dataset, I felt that the original dataset had too many non-purebloods in Slytherin and according to reasons I will state later on in this report, I felt like the initial ratio was unreasonable. Therefore, I feature engineered a 'prejudice factor' for Slytherin students and used weighted K-Means clustering to sort characters in such a way that shows that Slytherin is prejudiced against non-purebloods.

2 Methodology

2.1 Part One: Data Preparation and Preprocessing

What I checked and transformed before using the dataset is the following:

2.1.1 Checking for NAs and outliers. After downloading the dataset as a Pandas dataframe, I checked and found that there were no missing values in the dataframe and all the ordinal values were in range.

2.1.2 Checking datatypes. The table below shows the results. It can be seen that this aligns with what I had expected and does not need any modifications.

Table 1: Dataset Variables and Types

Variable	Type
Blood Status	Object
Bravery	Integer
Intelligence	Integer
Loyalty	Integer
Ambition	Integer
Dark Arts Knowledge	Integer
Quidditch Skills	Integer
Dueling Skills	Integer
Creativity	Integer
House (target variable)	Object

2.1.3 Transforming nominal values into numerical values. Since I want to train a decision tree, I need to transform all attributes into integers. From Table 1, it can be seen that only 'Blood Status' needs to be transformed. I fit-transformed this column in the following way:

```
le = LabelEncoder()
df['Blood Status Encoded'] = le.fit_transform(df['Blood
Status'])
```

In this transformation, 'Muggle-born' was transformed into 0, 'Half-blood' into 1, and 'Pure-blood' into 2.

2.1.4 Choosing Attributes for classification. As stated in the ***About the Dataset*** part, I mentioned that the sorting hat is not truly politically correct and takes more than just 'Bravery', 'Intelligence', 'Loyalty', and 'Ambition' into account when sorting students. Therefore I wanted to use two DataFrames (df) for training. These dfs differ in terms of attributes used for training. The first df contains all attributes from the downloaded dataset. The second only has the politically correct attributes mentioned above.

Selecting all attributes to train the classifiers:

```
feature_cols = ['Blood Status Encoded', 'Bravery',
'Intelligence', 'Loyalty',
                'Ambition', 'Dark Arts Knowledge',
                'Quidditch Skills',
                'Dueling Skills', 'Creativity']
X = df[feature_cols].values
y = df['House'].values
```

Selecting only politically correct attributes to train the classifiers:

```
feature_cols = ['Bravery', 'Intelligence', 'Loyalty',
                'Ambition']
X = df[feature_cols].values
y = df['House'].values
```

2.2 Part Two: Training Classifiers

After creating the two dfs with different attributes, I split the data for training and testing in the following way:

```
# Split the data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)
```

2.2.1 Base Classifier. I used the majority class clasifier as the base classifier. The code for this is the following:

```
# Create and train majority class classifier
majority_clf = DummyClassifier(strategy='most_frequent',
random_state=42)
majority_clf.fit(X_train, y_train)
# Make predictions
y_pred = majority_clf.predict(X_test)
# Find the majority class
majority_house = unique[np.argmax(counts)]
print(f"\nMajority House: {majority_house}")
print(f"Strategy: Always predict '{majority_house}'")
```

As expected, the accuracy of the base classifier is quite low:

```
============================================================
HOUSE DISTRIBUTION AFTER TRAIN-TEST SPLIT
============================================================
Training set distribution:
  Gryffindor  : 158 (22.6%)
  Hufflepuff  : 176 (25.1%)
  Ravenclaw   : 181 (25.9%)
  Slytherin   : 185 (26.4%)
Test set distribution:
  Gryffindor  :  68 (22.7%)
  Hufflepuff  :  75 (25.0%)
  Ravenclaw   :  77 (25.7%)
  Slytherin   :  80 (26.7%)
============================================================
MAJORITY CLASS CLASSIFIER
============================================================
Majority House: Slytherin
Strategy: Always predict 'Slytherin'
Accuracy: 0.2667 (26.67%)
```

2.2.2 Decision Tree Classifier. I trained a decision tree from the two aforementioned dfs. I used RandomizedSearchCV during the hyperparameter-tuning process. The parameters I used are the following:

```
# Define parameters for Decision Tree
dt_param_dist = {
'max_depth': [3, 5, 7, None],
'min_samples_split': [2, 5, 10, 15, 20],
'min_samples_leaf': [1, 2, 4, 8, 10],
'criterion': ['gini', 'entropy'],
'splitter': ['best', 'random'],
'max_features': ['sqrt', 'log2', None]
}
```

The process of using RandomizedSearchCV to find the best hyperparameters:

```
# Perform RandomizedSearchCV
dt_random_search = RandomizedSearchCV(
    estimator=dt_base,
    param_distributions=dt_param_dist,
    n_iter=10,
    cv=3,
    scoring='accuracy',
    random_state=42,
    n_jobs=-1,
    verbose=1
)
```

I used a low number of iterations and cross-validation because I felt that this amount was enough to create a model with an acceptable accuracy score.

2.2.3 Random Forest Classifier. Everything I did here was similar to what I did for the decision tree. The only difference is the parameters I set for hyperparameter-tuning. This is what I used instead:

```
# Define parameters for Decision Tree
param_dist = {
'n_estimators': [50, 100, 200, 300, 500],
'max_depth': [3, 5, None],
'min_samples_split': [2, 5, 10, 15],
'min_samples_leaf': [1, 2, 4, 8],
'max_features': ['sqrt', 'log2', None],
'bootstrap': [True, False],
'criterion': ['gini', 'entropy']
}
```

2.3 Part Three: Results From Classifier Algorithms

Table 2: Classifier Performance Comparison

Model	Attr.	Test Acc.
Decision Tree	All	98.00%
Random Forest	All	99.00%
Decision Tree	PC	92.67%
Random Forest	PC	95.67%

From the results in Table 2, it can be seen that the Random Forest Classifier trained from the df with all attributes performed the best on the test data, with an accuracy score of 99 %. Therefore, the results in the remaining part of this section will be about this instance.

Table 3: Confusion Matrix - RF (with all attributes)

Act./Pred.	Gryf.	Huff.	Rav.	Sly.
Gryffindor	68	0	0	0
Hufflepuff	2	73	0	0
Ravenclaw	0	1	76	0
Slytherin	0	0	0	80

Table 4: Feature Importance - RF (with all attributes)

Feature	Importance
Dark Arts Knowledge	0.229
Dueling Skills	0.218
Creativity	0.128
Loyalty	0.126
Intelligence	0.107
Bravery	0.091
Ambition	0.081
Quidditch Skills	0.020
Blood Status	0.000

Parts of the results from the classifiers coincide with the lore, whereas other parts do not.
The non-problematic part:
The fact that the models performed better when using all attributes for training is a good thing. This can be interpreted that the sorting hat is biased and takes in factors other than the politically correct attributes. This is in accordance with the Harry Potter lore.
The problematic part:
From table 4, it can be seen that the blood status plays no role in classifying the characters into their houses. This is not in accordance with the Harry Potter lore since Slytherins favor pure-bloods. This is where the second part of my project comes in; to "re-sort" these characters in a more lore-accurate way.

2.4 Part Four: Feature Engineering Prejudice of Slytherins

I had tried many ways of re-clustering the data but the way that worked best was to do K-Means clustering using some attributes plus feature engineered attributes. Therefore this will be the structure of my report.

My idea of the Slytherin Prejudice is comparable to when a woman wants to apply for a job when compared to a man - the woman would have to be extraordinary at something whereas a man could just be good at it. In this case the non-pure bloods would have to show signs that they are very suitable to be in Slytherin as compared to pure-bloods. To be able to implement this, I needed to first identify attributes that had strong correlations with slytherin characters, then put a non-pure-blood-penalty on these attributes.

2.4.1 finding attributes correlated with being Slytherin. I identified attributes that had strong correlations with slytherin characters by doing a conditional probability plot that shows how likely a character is to be sorted into Slytherin given there attribute scores (see appendix figure 1). **The results show that having high ambition and dark arts knowledge makes a student more likely to be sorted into Slytherin.**

2.4.2 adding the prejudice factor. I did this the following way:

```
# SLYTHERIN PREJUDICE FEATURE ENGINEERING
def calculate_prejudice_factor(row):
    """
    Calculate prejudice factor based on House and Blood
    Status
    only for the Slytherin house.
    """
    # define prejudice factors
    if row['House'] == 'Slytherin':  #--(1)
        if row['Blood Status'] == 'Pure-blood':
            return 1.0  #--(2)
        elif row['Blood Status'] == 'Half-blood':
            return 0.7  #--(3)
        elif row['Blood Status'] == 'Muggle-born':
            return 0.5  #--(4)
    return 1.0 # No prejudice for other houses

# Apply the prejudice factor and create new columns
prejudice_factor = df.apply(calculate_prejudice_factor,
axis=1)
df['slytherin_prejudice_ambition'] = df['Ambition'] *
prejudice_factor #--(5)
df['slytherin_prejudice_dark_arts'] = df['Dark Arts
Knowledge'] * prejudice_factor #--(6)
```

#–(1) means that I am applying this penalty only to Slytherins #–(2), (3), and (4) means if the Slytherin is a pure-blood they face no prejudice, if they are half-blood they face a '30%' prejudice, and if they are muggle-born they face a '50%' prejudice.
#–(5) and (6) shows that I am implementing this factors to the attributes 'Ambition' and 'Dark Arts Knowledge' only. Again, this means this prejudice only affects the probability of students getting sorted into Slytherin, since the prejudice is only applied to these two attributes, which increases the probability of getting sorted into Slytherin.

2.5 Part Five: K-Means Clustering

I decided to use K-Means clustering since I already knew I wanted four clusters (each responding to a Hogwarts house).

2.5.1 Unweighted K-Means Clustering with the original dataset. this serves as a baseline for comparision with the other clustering methods I tried. The attributes I used for clustering are:

```
cluster_features = ['Bravery', 'Intelligence', 'Loyalty',
'Ambition',
                    'Dark Arts Knowledge', 'Quidditch
                    Skills',
                    'Dueling Skills', 'Creativity','Blood
                    Status Encoded']
```

Despite the fact that I used 'Blood Status Encoded' to perform the clustering, the ratio of non-pure bloods in Slytherin is $\frac{177}{265} = 0.668$ and the Slytherin cluster is very well mixed between all the blood statuses (see appendix figure 2).

2.5.2 Unweighted K-Means Clustering with feature-engineered columns. The attributes I used to do the K-Means clustering is as follows:

```
cluster_features = ['Bravery', 'Intelligence', 'Loyalty',
                    'slytherin_prejudice_ambition',
                    'slytherin_prejudice_dark_arts',
                    'Quidditch Skills',
                    'Dueling Skills', 'Creativity', 'Blood
                    Status Encoded']
```

Despite using the feature engineered columns and 'blood status encoded', the ratio of non-pure bloods in Slytherin is still high ($\frac{132}{222} = 0.595$). From the PCA projection (see appendix figure 3), it can be seen that there is some blood status segregation inside the Slytherin cluster itself. However, the fact that this segregation happens *inside* the Slytherin cluster, and not that this segregation causes datapoints to be clustered into another house is unacceptable.

2.5.3 Weighted K-Means Clustering with feature-engineered columns. Since feature engineering prejudice was not enough to seperate non-pure bloods in Slytherin to an extent I though was lore-accurate, I decided to apply weights to the column 'blood status encoded' in the following way:

```
# Prepare features
cluster_features_dynamic = ['Blood Status
Encoded','Bravery', 'Intelligence',
                            'Loyalty',
                            'slytherin_prejudice_ambition'
                           'slytherin_prejudice_dark_arts',
                            'Quidditch Skills',
                           'Dueling Skills', 'Creativity']

# Create dynamic weight for Blood Status based on Ambition
and Dark Arts
base_weight = 1.0  # normal weight before scaling
scaling_factor = 0.44  # Maximum additional weight

# Calculate per-instance blood status weight
# Increasing the value of 'blood status weight' if
character has
# high weighted ambition and weighted dark arts knowledge
blood_status_weights = base_weight + scaling_factor *
(slytherin_prejudice_ambition_norm *
slytherin_prejudice_dark_arts_norm)

# Apply dynamic weights to Blood Status column (index 0)
X_dynamic_weighted[:, 0] = X_dynamic[:, 0] *
blood_status_weights #--(1)
```

This code contains the essence, there are other line in between these in the Jupyter notebook. However, this code shows how I implemented the weights to 'blood status encoded' (#–(1)). First, I defined 'blood-status-weights' which will be added to the values of 'blood status encoded'. I made 'blood-status-weights' to be dependent on the prejudiced ambition and prejudiced dark arts knowledge score because I believe that in this way, the blood-status-weights would only affect characters where the values of these two column are high, which would mean this would segregate the pure-blood Slytherins from the other instances. I was too correct with this idea; after clustering, the ratio of non-pure bloods in Slytherin was $\frac{0}{89} = 0$. From the PCA projection (see appendix figure 4), it can be seen that there is no blood status segregation in the Slytherin house since there are only pure bloods. This result is too extreme because in the Harry Potter books, there were half-bloods in Slytherin (Voldemort himself was a half blood). I am not satisfied with this result either.

2.5.4 Discussion and an interesting situation. From sections 2.5.2 and 2.5.3, it can be seen that my results fall between two extremes; either there are many non-pure bloods in Slytherin or none at all. I tried experimenting with the values of both the Slytherin Prejudice Factor (2.4.2) and the scaling-factor value for the blood-status-weights (2.5.3). The result is that there seems to be a threshold value that causes this switch and there is no middle ground for the ratio of non-pure bloods in Slytherin. For example, if the prejudice factor is set to 0.7 and 0.3 (the one from the code in 2.4.2) and if the scaling factor is 0.44, the ratio of non-pure bloods in Slytherin is 0, however, if the scaling factor is set to 0.43 or below, the ratio jumps to approximately 0.6 (see appendix figure 4 and 5). However, an interesting situation arose when I made a mistake during the coding process. If I apply the blood-status-weights with these differences:

```
# Prepare features
cluster_features_dynamic = ['Bravery', 'Intelligence',
'Loyalty',
                            'slytherin_prejudice_ambition',
                           'slytherin_prejudice_dark_arts',
                            'Quidditch Skills',
                            'Dueling Skills', 'Creativity']

scaling_factor = 50  # Maximum additional weight

# Apply dynamic weights to Blood Status column (index 0)
X_dynamic_weighted[:, 0] = X_dynamic[:, 0] *
blood_status_weights
```

This means I am applying the blood-status-weights to 'Bravery' instead of 'Blood Status Encoded'. Strangely, after clustering, the ratio of non-pure bloods in Slytherin was $\frac{23}{109} = 0.211$. From the PCA projection (see appendix figure 6), it can be seen that there is some segregation in the Slytherin house. This is the results I tried to obtain. I hypothesize that this method worked because I applied some modifications to attributes that did not correlate with just Slytherin, which made the results not so extreme (like in 2.5.3). However, this method was simply an error and does not make sense in my mind.

3 Appendix

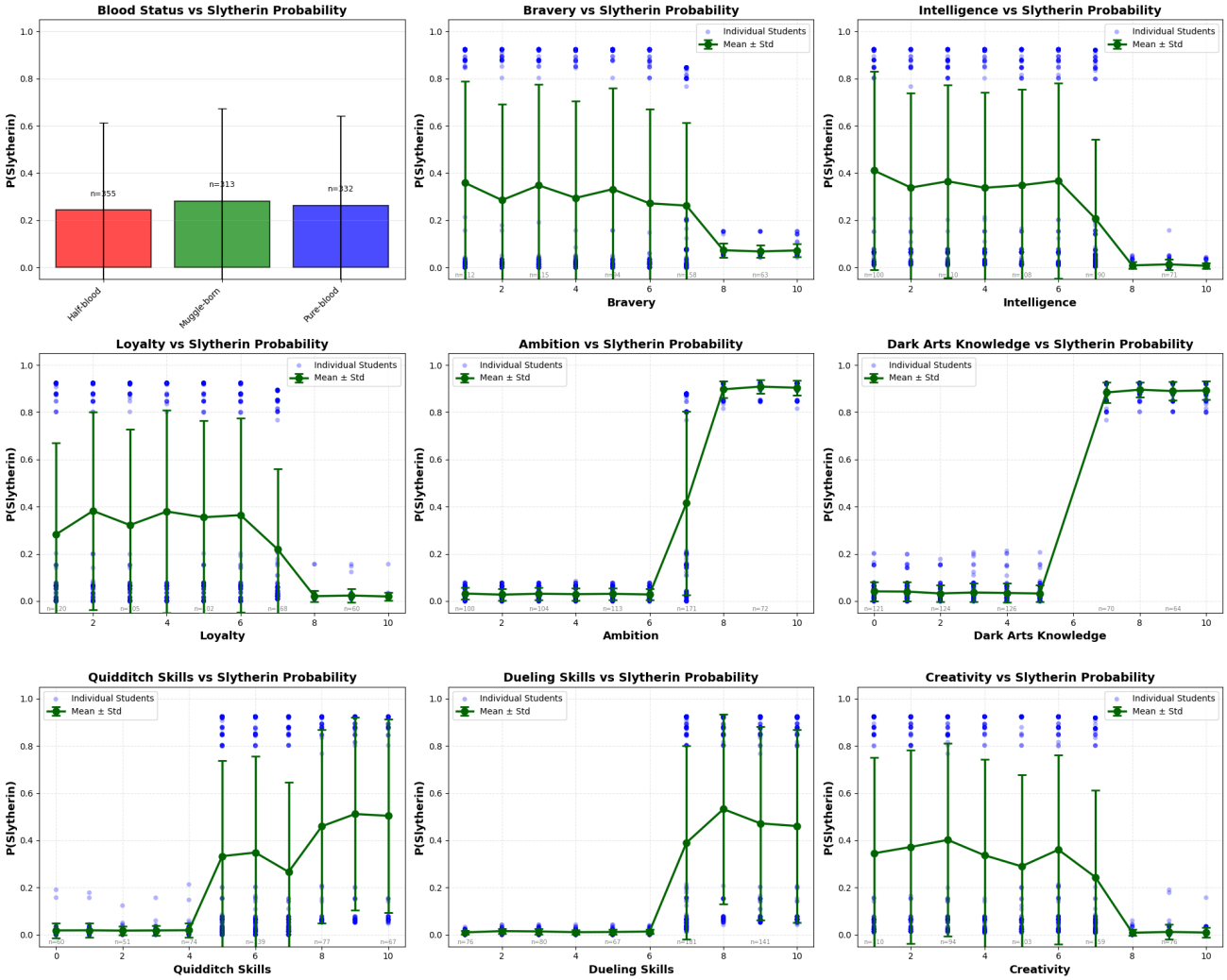

Figure 1: Conditional Probability Plots showing the probability of students being sorted into Slytherin given values of some attribute

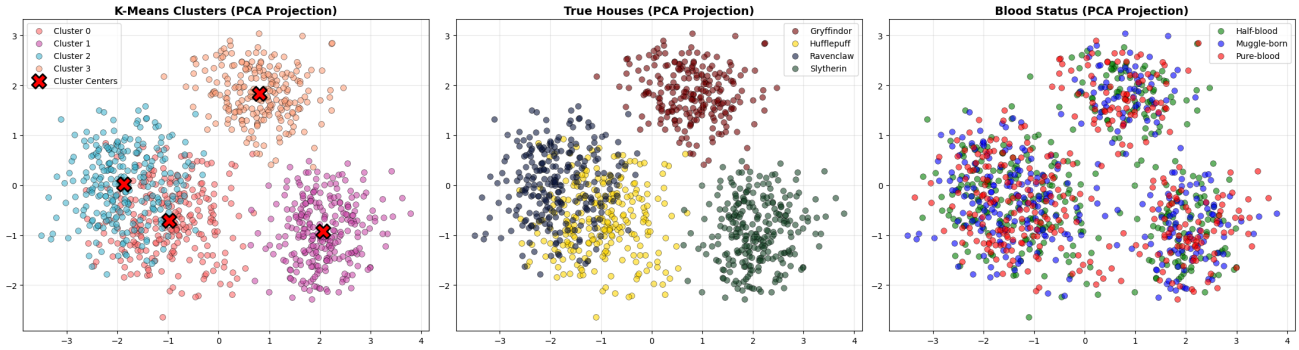

Figure 2: PCA Projection Comparison of clusters from the original dataset: K-Means Clusters (left), True Houses (center), and Blood Status (right)

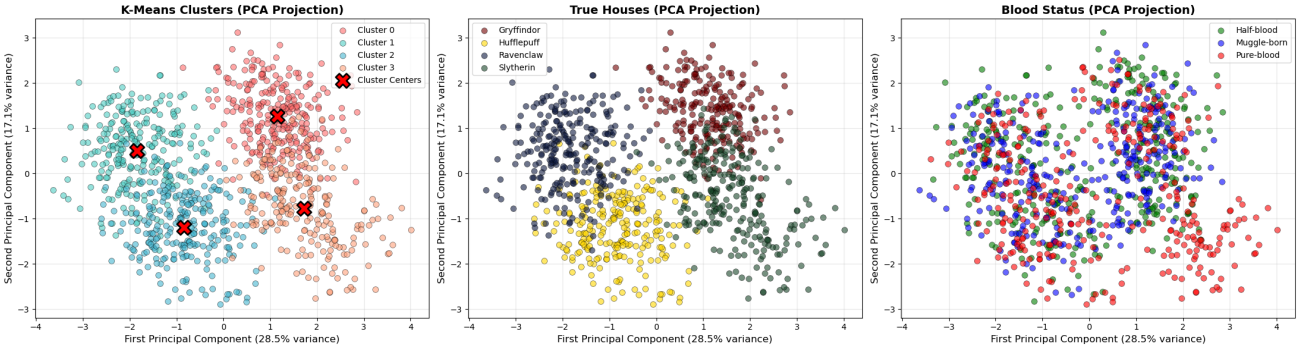

Figure 3: PCA Projection Comparison of clusters when using feature engineered columns: K-Means Clusters (left), True Houses (center), and Blood Status (right)

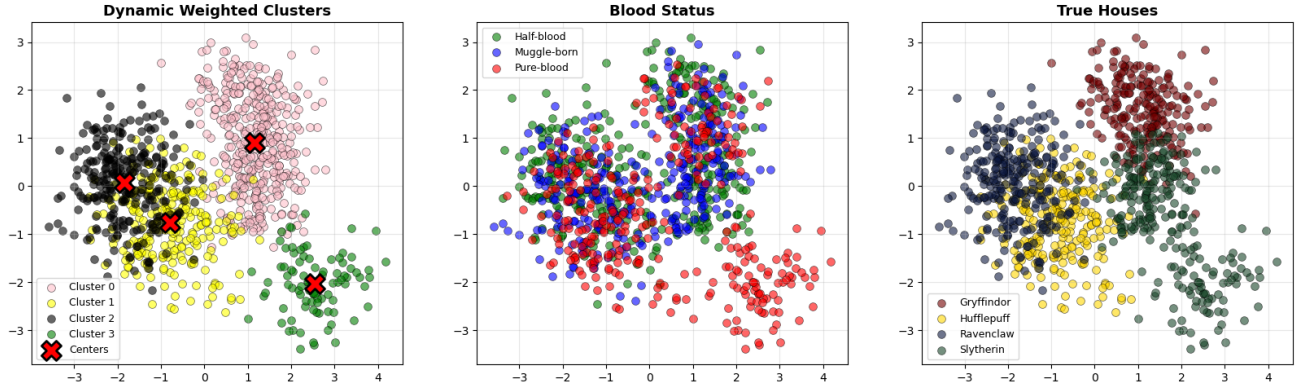

Figure 4: PCA Projection Comparison of clusters when using feature engineered columns and weighted blood status (scaling factor 0.44): K-Means Clusters (left), True Houses (center), and Blood Status (right)

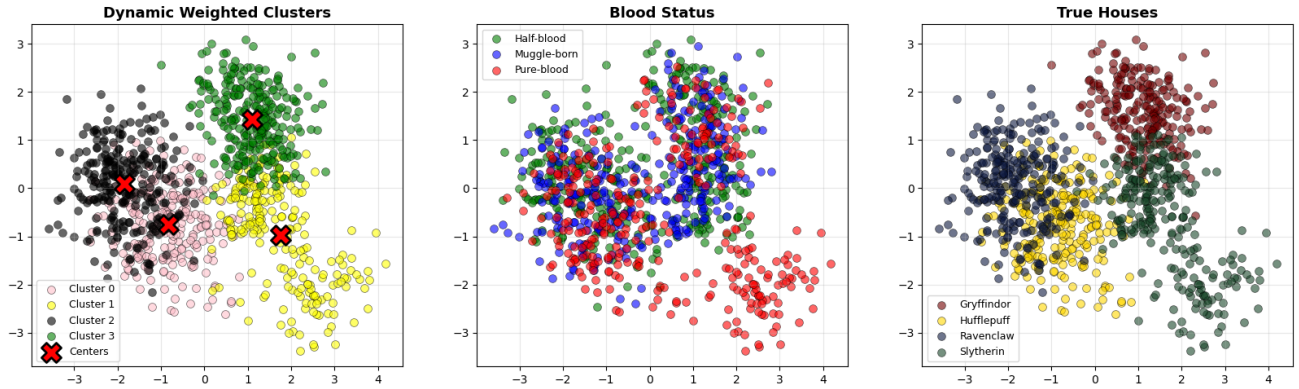

Figure 5: PCA Projection Comparison of clusters when using feature engineered columns and weighted blood status (scaling factor 0.43): K-Means Clusters (left), True Houses (center), and Blood Status (right)

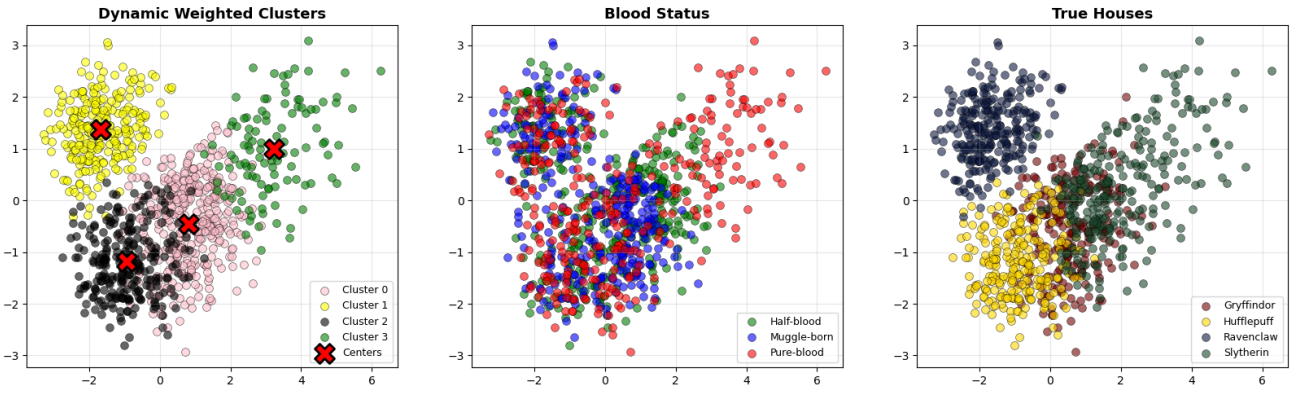

Figure 6: PCA Projection Comparison of clusters when using feature engineered columns and weighted bravery score: K-Means Clusters (left), True Houses (center), and Blood Status (right)